

Formalization of the TCG MARS Specification in Rocq

Junguo Xu

Advisor: Julian Pritzi

Chair of Computer Systems

<https://dse.in.tum.de/>



25.05.2026 – 25.09.2026

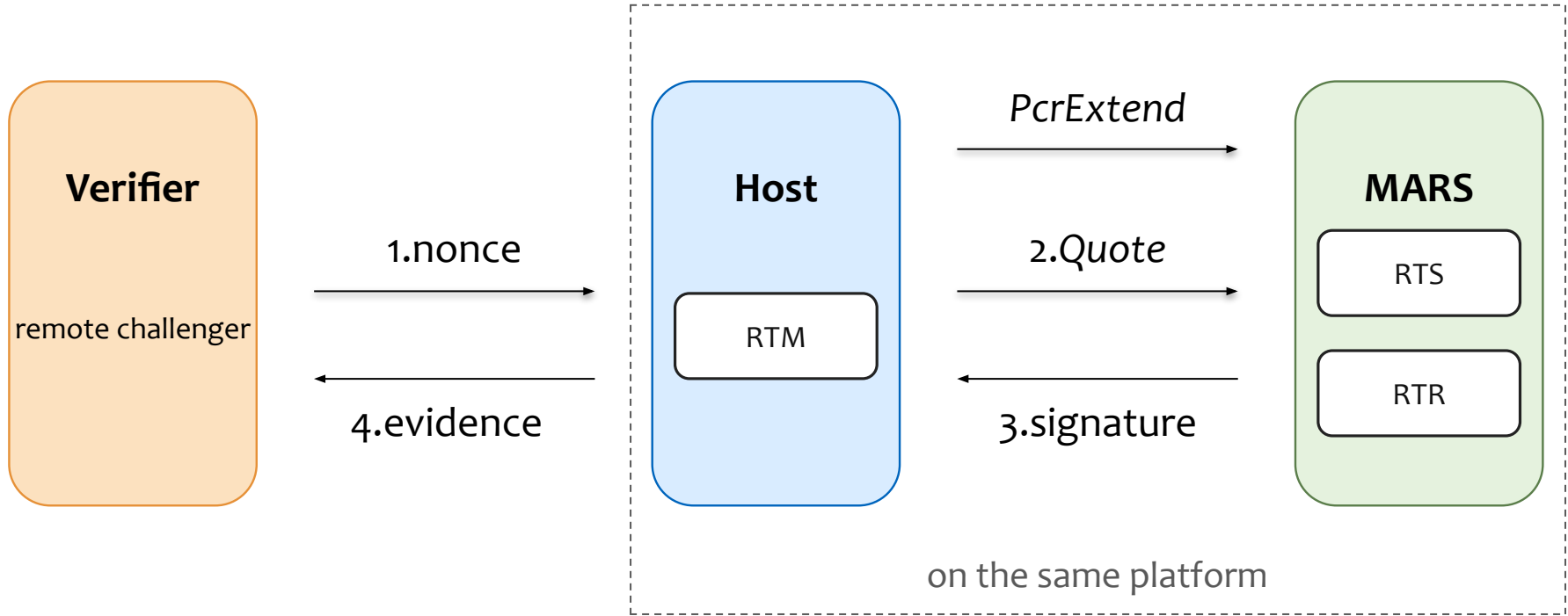
Background



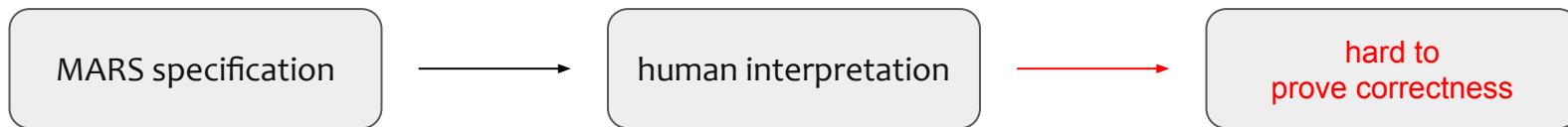
- Remote attestation is important to modern systems
- Different specifications provide RoT (Root of Trust)
 - TPM
 - MARS
- Most specifications are written in natural language
 - ambiguous
 - missing assumptions
 - inconsistent



Background



- Evaluating specifications often relies on manual review
- Many other specifications have formal models (RISC-V ISA, WebAssembly)



There is no precise, machine-checkable model of core MARS command behavior

How can we formally model the MARS state machine in Rocq to identify inconsistencies and prove invariants of the MARS specification?

Design Goals

Faithfulness

The behavior of the command interface follows MARS

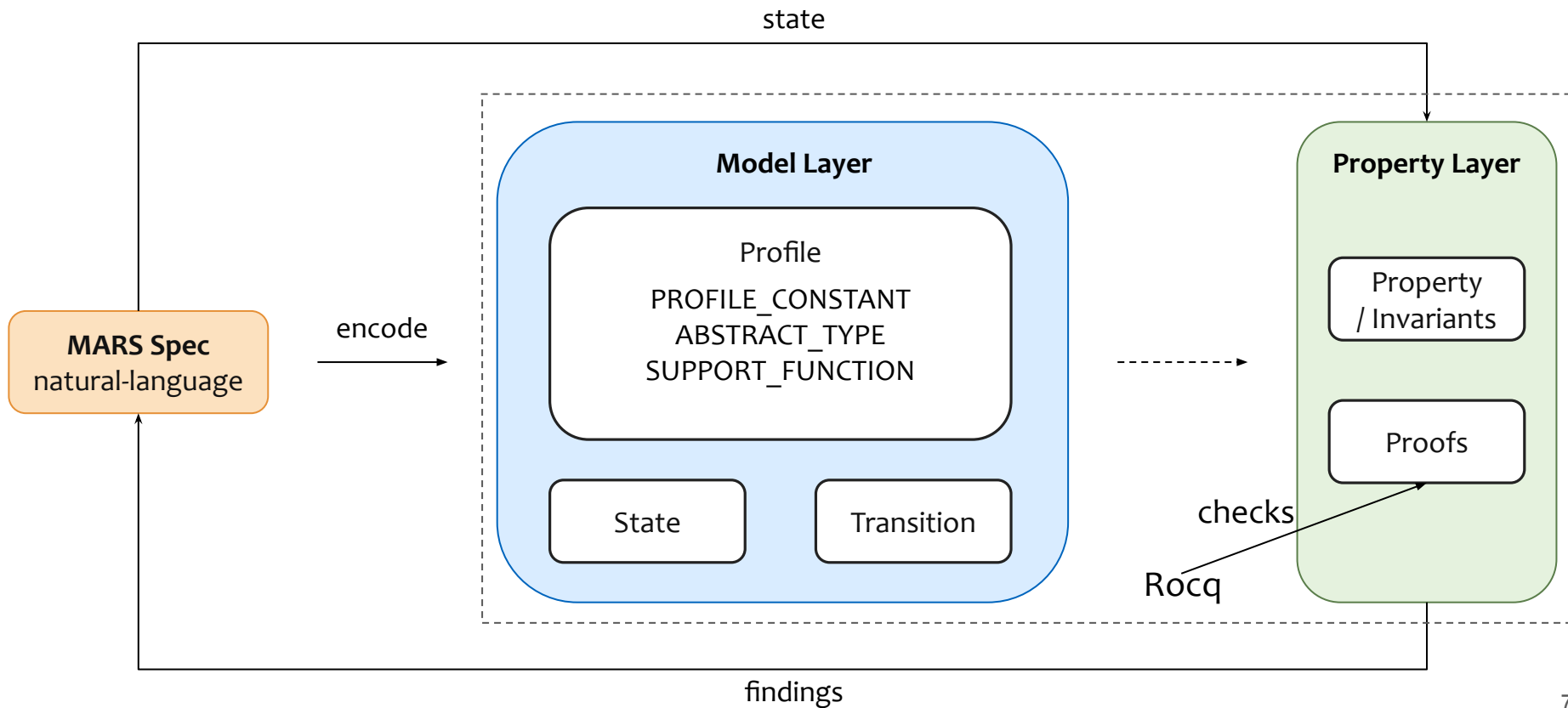
Generality

The formalization can be applied to all valid profiles

Totality

The formalization has a defined behavior for all reachable states

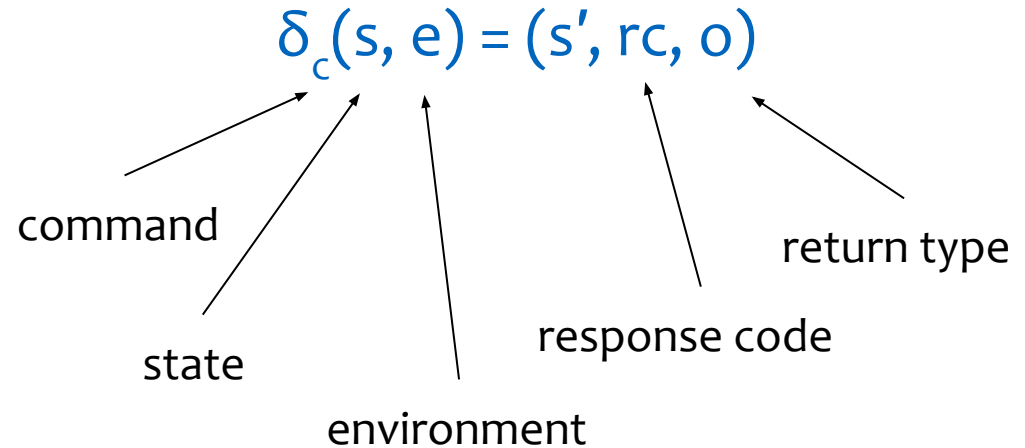
System Overview



MARS_STATE :

Field	Meaning
failure	Failure-mode flag
PS	Persistent primary secret
DP	Current derivation parent
regs	PCR and TSR values
shc	Optional hash sequence

Transition :



Example Command: MARS_Quote

Inputs :

- Register selection
- Nonce
- Context

Outputs :

- Signature

Behavior :

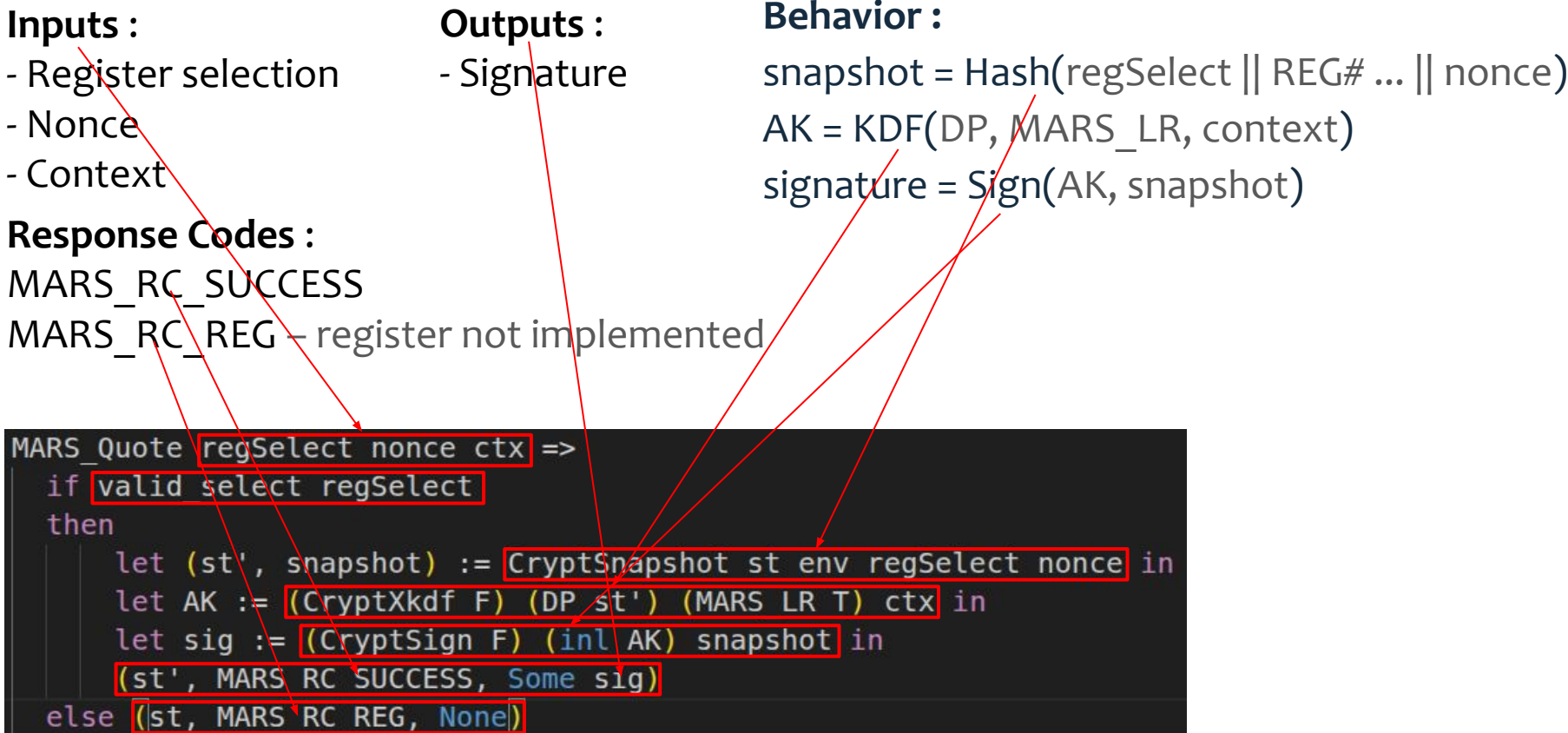
snapshot = Hash(regSelect || REG# ... || nonce)
AK = KDF(DP, MARS_LR, context)
signature = Sign(AK, snapshot)

Response Codes :

MARS_RC_SUCCESS

MARS_RC_REG – register not implemented

```
MARS_Quote regSelect nonce ctx =>
  if valid select regSelect
  then
    let (st', snapshot) := CryptSnapshot st env regSelect nonce in
    let AK := (CryptXkdf F) (DP st') (MARS_LR T) ctx in
    let sig := (CryptSign F) (inl AK) snapshot in
    (st', MARS_RC SUCCESS, Some sig)
  else (st, MARS_RC REG, None)
```



Properties

State invariants

PS persistence

Register count

DP update rule

Reachable invariants

Command semantics

Failure-mode lockout

Self-test failure

Sequence discipline

Registers

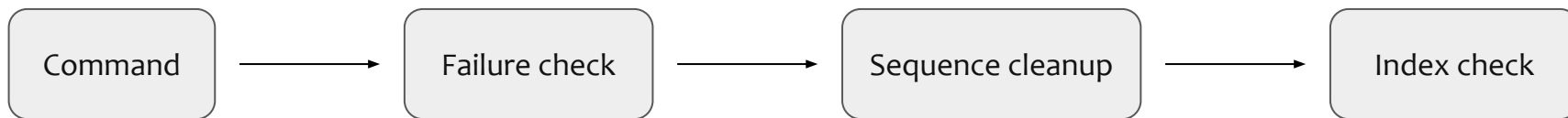
PCR / TSR discipline

Invalid-index rejection

ps_persistent : $\forall c, s, e : \delta_c(s, e) = (s', rc, o) \Rightarrow s[PS] = s'[PS]$.

They hold for all admissible profiles

- The MARS specification does not specify the check ordering



We found some contradictions in the Normative C code and TCG emulator!

- Missing failure check in the C code
- *CapabilityGet* is rejected in failure mode in the emulator

Conclusion

Model coverage

A profile-parametric state machine with all 13 commands are modeled

Machine-checked properties

Nine groups of properties and invariants fully proved in Rocq

Specification findings

Three findings and five caveats are documented

The model can help clarify the specification and correct the emulator

thank you for your attention !

Case	Model decision
F1 Failure + invalid index	RC_FAILURE before index checks
F2 Failure + active sequence*	Reject without cleanup
F3 Invalid non-sequence command	Clear shc, then RC_REG

Area	Observation
C1 CapabilityGet	Clears shc in an arbitrary failure state
C2 Normative C snippets	Need global failure and sequence checks
C3 DpDerive(NULL)	Validates regSelect before resetting DP
C4 SignatureVerify	RC_SUCCESS can accompany result = false
C5 TCG emulator*	CapabilityGet lockout and ignored init self-test